Interactive Implied Volatility Surfaces for Single-Stock Equity Options

Kevin Williams

Project proposal — Numerical and Symbolic Computation
October 16, 2026

Abstract

This project builds an interactive implied volatility surface for single-stock equity options in MATLAB, applying the symbolic and numerical methods from the course to a real financial model. Option prices quoted in the market imply a different volatility for each strike and expiry, a pattern that the Black–Scholes assumption of constant volatility cannot explain; the surface makes that structure visible. The project is an analysis tool for understanding the model's limits, and it extends an existing Black–Scholes–Merton engine written in Python with an OCaml compute backend. A Python exporter pulls a full options chain for a chosen ticker, filters out stale and illiquid quotes, and saves a timestamped snapshot that MATLAB loads directly. In MATLAB, the Symbolic Math Toolbox derives the Black–Scholes price and Greeks and generates a numerical Newton–Raphson implied-volatility solver with a bracketing fallback, and the resulting volatilities are cross-checked against the OCaml engine. A smooth surface is built from the out-of-the-money quotes by scattered-data interpolation, and an App Designer interface lets the user select a ticker and expiry and view the three-dimensional surface alongside smile and term-structure slices.

1 Motivation and goals

The Black–Scholes model prices a European option from five inputs: spot price S, strike K, time to expiry T, risk-free rate r and a single constant volatility σ. Inverting the model, i.e. solving for the σ that reproduces an observed market price, gives the *implied volatility* $\sigma_{\text{imp}}(K, T)$. If the model were right, σ_{imp} would be the same for every option on a given stock. In practice it varies with strike (the *smile* or *skew*) and with maturity (the *term structure*), and plotting σ_{imp} against both gives the *volatility surface*.

The project has three goals:

1. Build a reusable pipeline from a live options chain to a clean, validated set of implied volatilities, for any single-stock ticker.
2. Use the course's tools on a real problem: symbolic differentiation and code generation, root finding with safeguards, and interpolation of scattered data.
3. Verify the MATLAB results independently against the existing Python/OCaml engine, so that correctness is demonstrated rather than assumed.

2 System overview

The work is split by strength. Python and OCaml, which already exist, handle data acquisition and provide an independent reference implementation; MATLAB handles the mathematics and the interactive display. The two sides communicate through a plain CSV snapshot, so the MATLAB code can be run (and graded) without the Python stack and works on any file that follows the column format.

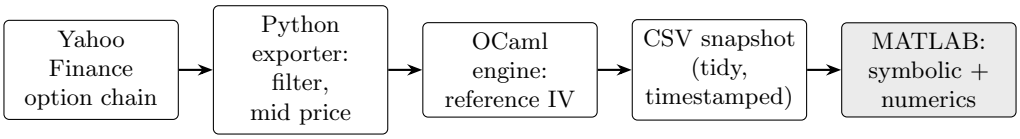


Figure 1: Data flow. The shaded stage is the focus of this project; the earlier stages already exist and are extended by a snapshot exporter. The interactive application sits on top of the MATLAB stage.

3 Methods

3.1 Symbolic derivation

With a continuous dividend yield q (zero reproduces the existing engine exactly), the call and put prices are

$$d_1 = \frac{\ln(S/K) + (r - q + \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}, \qquad d_2 = d_1 - \sigma\sqrt{T}, \tag{1}$$

$$C = Se^{-qT}\Phi(d_1) - Ke^{-rT}\Phi(d_2), \qquad P = Ke^{-rT}\Phi(-d_2) - Se^{-qT}\Phi(-d_1), \tag{2}$$

where Φ is the standard normal distribution function. The Symbolic Math Toolbox differentiates these expressions to obtain the Greeks, in particular vega $\nu = \partial C/\partial\sigma = Se^{-qT}\varphi(d_1)\sqrt{T}$, and `matlabFunction` compiles them into vectorised numerical functions. Two identities are checked on the symbolic expressions rather than assumed: the Black–Scholes partial differential equation,

$$-\frac{\partial C}{\partial T} + (r - q)S\frac{\partial C}{\partial S} + \tfrac{1}{2}\sigma^2 S^2\frac{\partial^2 C}{\partial S^2} - rC = 0, \tag{3}$$

and put–call parity, $C - P = Se^{-qT} - Ke^{-rT}$.

3.2 Implied volatility solver

Given a market price C_{mkt}, the solver finds σ with $C(\sigma) = C_{\mathrm{mkt}}$ by Newton–Raphson,

$$\sigma_{n+1} = \sigma_n - \frac{C(\sigma_n) - C_{\mathrm{mkt}}}{\nu(\sigma_n)}, \tag{4}$$

using the symbolically derived vega. Because the price is strictly increasing in σ, every iterate also tightens a bracket $[\sigma_{\mathrm{lo}}, \sigma_{\mathrm{hi}}]$ (initially $[10^{-4}, 5]$). If vega is nearly zero or a Newton step leaves the bracket, the step falls back to bisection, which guarantees convergence. A price outside the

attainable range, such as one below intrinsic value, returns `NaN` because it violates no-arbitrage bounds. The method records how often the fallback was needed, which gives a direct comparison of the two root-finding strategies.

A practical limit is *identifiability*: the volatility error is roughly the price tolerance divided by vega, so deep in-the-money and short-dated options, whose prices are almost independent of σ, cannot yield a reliable σ_{imp}. This motivates the choice of quotes below.

3.3 Forward price and quote selection

Put–call parity implies $F = K + e^{rT}(C - P)$ for any strike with both a call and a put. For each expiry the forward F is taken as the median over the five strikes nearest spot. Only out-of-the-money quotes are used (puts with $K < F$, calls with $K \geq F$), since they carry mostly time value and therefore the most reliable volatility information. Positions are expressed in log-moneyness $k = \ln(K/F)$.

3.4 Surface construction

The surface is interpolated in total variance $w(k, T) = \sigma_{\text{imp}}^2 T$, which behaves more regularly across maturities than σ_{imp} itself. Both axes are rescaled to $[0, 1]$, because k and T differ in scale by an order of magnitude and Delaunay triangulation is scale-sensitive. The interpolant is piecewise linear on the triangulation, with volatility recovered as $\sigma_{\text{imp}} = \sqrt{w/T}$. The surface is deliberately *not* extrapolated: outside the convex hull of the data it is undefined, so holes mark regions with no market information. Slices then follow directly: a *smile* fixes T and varies k, and a *term structure* fixes k (by default $k = 0$, at the forward) and varies T.

3.5 Interactive interface

An App Designer application will provide a ticker field with a refresh button that runs the Python exporter through MATLAB's Python interface, an expiry selector, the three-dimensional surface with the market quotes overlaid, and linked smile and term-structure panels. Cached snapshots keep the demonstration reproducible when markets are closed or the data feed is unavailable.

4 Validation

Correctness is checked at three levels, by automated MATLAB tests where possible.

- **Symbolic checks:** the PDE residual and put–call parity are verified symbolically and numerically at random points.
- **Known-answer tests:** prices generated from known volatilities are inverted to recover them; the surface interpolator is tested against an analytic surface; invalid inputs must be rejected with clear errors.
- **Cross-validation:** MATLAB implied volatilities are compared with the OCaml engine on live data.

Preliminary results from the completed components are in Table 1. The maximum difference to the OCaml engine, 5×10^{-7}, is exactly the rounding error of that engine's output, which is printed to six decimal places; the two solvers therefore agree to the precision the engine reports.

Check	Result
Symbolic PDE and put–call parity	proved symbolically; residual 2×10^{-14} numerically
Implied-volatility round trip (142 well-posed cases)	worst error 3.1×10^{-7}
AAPL snapshot, 12 Oct 2026: 140 OTM contracts, 8 expiries	all converged, no bisection fallback needed
Difference from OCaml engine on that snapshot	median 2.6×10^{-7}, maximum 5.0×10^{-7}
Interpolation error on an analytic surface	below 5×10^{-4} (volatility space), 3.5×10^{-3} (total variance)

Table 1: Preliminary validation results.

5 Scope, limitations and extensions

Limitations. Listed equity options are American-style, so early-exercise value is ignored by the European model; this matters most for in-the-money options and is one more reason for the out-of-the-money selection. The engine has no dividend term ($q = 0$), so the forward is inferred from put–call parity instead. Free market data is delayed and noisy: thinly quoted, short-dated expiries can produce spurious dips in the surface, and a minimum-quotes-per-expiry rule is under consideration. Finally, the interpolated surface is not guaranteed to be free of arbitrage.

Extensions (outside this project). Fitting a parametric smile model to each expiry, such as the SVI parameterisation, together with explicit no-arbitrage checks on strike (butterfly) and maturity (calendar) convexity, would be a natural follow-up and would use this interpolated surface as its baseline.

6 Status

Component	Status
Snapshot exporter and CSV format (Python)	Complete
Symbolic derivation and implied-volatility solver	Complete
Forward estimation and out-of-the-money selection	Complete
Surface interpolation, slices and automated tests	Complete
Interactive App Designer application	Planned
Written report and demonstration	Planned